

Tài liệu để build P0

Ngày note: 2026-06-13

Mục tiêu P0: dựng được luồng end-to-end tối thiểu:

```
video/keyframe -> manifest -> index -> search API -> UI grid ->
preview/frame neighbor -> dry-run submit -> log/eval
```

P0 không cần agent, CRAG, VLM rerank, fine-tune, hay DRES full. Cần hệ thống chạy được trước.

Nếu cần bản chi tiết để code ngay, đọc thêm:

- [P0 Implementation Spec](#)
- [P0 Task Cards Cho Team](#)

Nguyên tắc kiến trúc: build theo OOP/adapter để dễ đổi stack

P0 là baseline để lắp tiếp P1/P2, nên code không được dính cứng vào một model hay một vector DB cụ thể. Viết theo interface/adapter để sau này đổi CLIP -> SigLIP/PE, FAISS -> Qdrant, dry-run submit -> BTC submit mà không phải viết lại toàn bộ hệ thống.

Các module nên có interface rõ:

QueryClassifier

- RuleBasedQueryClassifier
- LlmQueryClassifier

TaskStrategy

- TextualKisStrategy
- VisualKisStrategy
- QaStrategy
- TrakeStrategy
- OcrHeavyStrategy
- AsrHeavyStrategy

FrameExtractor

- FfmpegFrameExtractor
- SceneDetectFrameExtractor

EmbeddingModel

- ClipEmbeddingModel
- SiglipEmbeddingModel
- PrecomputedNpyEmbeddingModel

VectorIndex

- FaissVectorIndex
- QdrantVectorIndex

TextSearchIndex

- Bm25TextSearchIndex
- QdrantSparseTextSearchIndex

Retriever

- VisualRetriever
- TextRetriever
- HybridRetriever

SubmissionClient

- DryRunSubmitClient

- DresSubmitClient
- BtcSubmitClient

Rule thực dụng:

- Business logic nhận interface, không nhận class cụ thể.
- Config quyết định dùng implementation nào.
- Response schema giữ ổn định để frontend không bị vỡ khi đổi backend stack.
- Manifest schema là contract trung tâm, không phụ thuộc model/index.
- Mỗi adapter có test hoặc smoke test riêng.

Ví dụ flow đúng:

```
SearchService -> HybridRetriever -> VectorIndex interface ->  
FaissVectorIndex hoặc QdrantVectorIndex
```

Flow nên tránh:

```
/search endpoint gọi thẳng faiss.Index.search(...) rồi tự format mọi thứ  
trong route
```

Lý do: baseline P0 là cái khung để gắn thêm OCR, ASR, TRAKE, QA, reranker, DRES/BTC submit. Nếu khung sạch, đổi stack là thay adapter. Nếu khung bẩn, mỗi lần đổi model là sửa cả backend lẫn frontend, rất dễ nổ sát deadline.

Nguyên tắc bài toán: mỗi dạng câu có strategy riêng

Không được coi mọi query là một bài `/search` giống nhau. Mỗi dạng câu có cách xử lý, cách rerank, UI payload và format submit khác nhau.

Flow nên là:

```
User query
-> QueryClassifier
-> chọn TaskStrategy
-> strategy gọi retriever/reranker phù hợp
-> strategy build UI payload
-> strategy format submission
```

Interface nên có:

```
TaskStrategy
- retrieve(query_context)
- rerank(candidates, query_context)
- build_ui_payload(results)
- format_submission(selected_candidate)
```

Các strategy chính:

Strategy	Cách xử lý riêng	Output
TextualKisStrategy	Visual/caption search, group theo video, boost cluster compactness, penalize intro	<video, frame>
VisualKisStrategy	Query thường mở hồ; dùng query template/rewrite, structured caption, frame neighbor/timeline	<video, frame>
QaStrategy	Tách retrieval question và answer question; extract answer từ OCR/ASR/caption/VLM	<video, frame, answer>
TrakeStrategy	Tách E1..En; retrieve từng event; DP/beam search theo cùng video và timestamp tăng dần	<video, frame1, frame2, ...>
OcrHeavyStrategy	Ưu tiên BM25/OCR exact match, visual dùng để validate context	<video, frame> hoặc <video, frame, answer>

AsrHeavyStrategy	Ưu tiên transcript/time alignment, chọn frame gần đoạn transcript	<video, frame> hoặc <video, frame, answer>
------------------	---	---

P0 chỉ cần implement chắc TextualKisStrategy và có stub/interface cho các strategy còn lại. Như vậy baseline không bị nhốt vào một kiểu search duy nhất, sau này lắp QA/TRAKE không phải bẻ cong code.

Tài liệu chính

Mảng P0	Tài liệu	Đọc để làm gì
Dataset/video structure	V3C paper , V3C1 Data.gov	Hiểu cấu trúc video -> shot/segment -> keyframe -> metadata
Cut keyframe	FFmpeg docs , PySceneDetect docs	Tự extract frame/shot, tạo thumbnail
Visual embedding	OpenAI CLIP GitHub	Encode image/text, làm search baseline
Vector search nhanh	FAISS getting started	Prototype local nhanh nhất
Vector DB có filter	Qdrant quickstart	Search + metadata filter tử tế hơn FAISS
Text search nhẹ	rank bm25	BM25 cho title/metadata/OCR/transcript
Backend API	FastAPI first steps	Dựng /search , /frames/{id}/neighbors , /submit/dry-run
Frontend	Vite guide	Dựng React UI nhanh
Submit/mock contest	DRES GitHub	Hiểu submit format/logging; P0 chỉ cần mock submit

Thủ tự đọc và build

1. Dataset + Manifest

Đọc:

- [V3C paper](#)
- [V3C1 Data.gov](#)
- [FFmpeg docs](#)
- [PySceneDetect docs](#)

Output cần có:

Raw mock data nên theo cấu trúc V3C-compatible:

```
data/mock_v3c/
  videos/<video_id>/<video_id>.mp4
  keyframes/<video_id>/*.jpg
  thumbnails/<video_id>/*.jpg
  msb/<video_id>.tsv
  info/<video_id>.json
```

Sau đó viết importer để chuyển raw layout này thành `manifest.jsonl`. Core backend/frontend chỉ đọc manifest, không hardcode path V3C.

```
{
  "video_id": "L01_V001",
  "frame_id": "L01_V001_000123",
  "timestamp": 12.4,
  "frame_path": "data/processed/frames/L01_V001/000123.jpg",
  "thumb_path": "data/processed/thumbs/L01_V001/000123.jpg",
  "video_path": "data/raw/videos/L01_V001.mp4",
  "source": "self_cut"
}
```

Done khi:

- Scan được video.
- Tạo được keyframe/thumbnail.
- Ghi được `manifest.jsonl`.
- Backend/UI có thể dùng path trong manifest.

2. Visual Search Baseline

Đọc:

- [OpenAI CLIP GitHub](#)
- [FAISS getting started](#)
- hoặc [Qdrant quickstart](#)

Output cần có:

- Encode toàn bộ keyframe thành vector.
- Build index.
- Encode text query.
- Search top-100 frame.

Done khi:

```
search("người đàn ông đứng cạnh xe máy") -> [frame candidates]
```

Mỗi candidate tối thiểu có:

```
{
  "video_id": "L01_V001",
  "frame_id": "L01_V001_000123",
  "timestamp": 12.4,
  "score": 0.73,
  "thumb_path": "...",
  "frame_path": "..."
}
```

3. Text Search Nhẹ

Đọc:

- [rank_bm25](#)

Output cần có:

- BM25 trên title/description/metadata.
- Sau này thay bằng OCR/transcript thật.

Done khi:

- Query tên riêng/số hiệu/từ khóa cụ thể có thể tìm bằng lexical search.
- Có thể fusion đơn giản với visual score.

4. Backend API

Đọc:

- [FastAPI first steps](#)

Endpoint P0:

```
GET    /health
POST   /search
GET    /frames/{frame_id}/neighbors
GET    /videos/{video_id}
POST   /submit/dry-run
```

Response `/search` nên có:


```
{
  "query": "người đứng cạnh xe máy",
  "took_ms": 82,
  "results": [
    {
      "rank": 1,
      "video_id": "L01_V001",
      "frame_id": "L01_V001_000123",
      "timestamp": 12.4,
      "score": 0.73,
      "thumb_url": "/media/thumbs/L01_V001/000123.jpg",
      "frame_url": "/media/frames/L01_V001/000123.jpg",
      "video_url": "/media/videos/L01_V001.mp4",
      "evidence": {
        "visual_score": 0.73,
        "text_score": null
      }
    }
  ]
}
```

5. Frontend UI

Đọc:

- [Vite guide](#)

UI P0 cần:

- Search box.
- Grid thumbnail.
- Click candidate -> detail panel.
- Preview frame/video.
- Frame trước/sau.
- Group by video nếu kịp.
- Nút dry-run submit.
- Query history/log đơn giản.

Done khi:

- User gõ query.
- Thấy top candidates.
- Click xem evidence.
- Dry-run submit trong ≤ 2 click.

6. Submit/Mock Contest

Đọc:

- [DRES GitHub](#)

P0 chưa cần dựng DRES full. Chỉ cần bắt chức submit:

```
POST /submit/dry-run
```

Body:

```
{
  "query_id": "q001",
  "video_id": "L01_V001",
  "frame_id": "L01_V001_000123",
  "answer": null,
  "elapsed_ms": 42100
}
```

Log lại:

```
query_id, video_id, frame_id, answer, elapsed_ms, created_at
```

Checklist P0

- ☐ Có dataset mẫu nhỏ.
- ☐ Có `manifest.jsonl`.
- ☐ Có keyframe/thumbnail.
- ☐ Có visual embedding index.
- ☐ Query text trả top-100 frame.

- ☐ Có FastAPI `/search` .
- ☐ Có UI grid thumbnail.
- ☐ Click candidate xem được frame/video.
- ☐ Xem được frame lân cận.
- ☐ Dry-run submit lưu log.
- ☐ Có 30-50 query eval nhỏ.
- ☐ Có script đo Recall@10/50/100 và latency.

Không làm trong P0

- LangGraph/CRAG.
- Multi-agent.
- Gemini/Qwen rerank.
- Wikidata/InfoSeek.
- Fine-tune model.
- TRAKE full.
- QA full.
- UI animation/đẹp hóa.

P0 là dựng đường ống. Khi nước chảy được rồi mới tính gắn turbine.